

ITA0612 – MACHINE LEARNING

CO1-AT1

APPLICATION BASED QUESTIONS

NAME: MONIKA R

REG.NO: 192324281

Submitted To:

Shri Vindhya A

Submitted on:

14-07-2026

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING)** Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – Application-Based Questions (3 Questions)

Course Code:	ITA0609	Course Title:	Machine Learning
CO Assessed:	CO1 – Analyze (BL 3) 40% of CIA	Assessment:	Assessment Tool 1 – Application Based Questions.
Weightage:		Total Marks:	30 (3 Questions ×10 Mark)
CO1 Statement:	<i>To acquire a foundational understanding of key machine learning concepts including version spaces, candidate elimination, inductive bias, and heuristic search (BL1)</i>		
Student Name:	Monika R	Reg. No.:	192324281
Section / Batch:	SLOT A	Date:	13.7.26

Code of Conduct

I (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Monika R

Signature of the Student:

QUESTION :1

A parking facility has three sections containing cars of different colors. Section A has 5 red, 3 blue, and 2 black cars; Section B has 4 red, 6 blue, and 5 black cars; and Section C has 7 red, 2 blue, and 6 black cars. A section is chosen at random, but Section B is twice as likely to be selected as each of the other sections due to higher usage. A randomly selected car from the chosen section turns out to be blue. What is the probability that the car came from Section C?

ANSWER:

Given:

Section	Red Cars	Blue Cars	Black Cars	Total Cars
A	5	3	2	10
B	4	6	5	15
C	7	2	6	15

Section **B** is twice as likely to be selected.

Therefore,

$$P(A) = \frac{1}{4}, P(B) = \frac{1}{2}, P(C) = \frac{1}{4}$$

Blue-car probabilities:

$$P(\text{Blue} | A) = \frac{3}{10}$$

$$P(\text{Blue} | B) = \frac{6}{15} = \frac{2}{5}$$

$$P(\text{Blue} | C) = \frac{2}{15}$$

Apply Bayes' Theorem

$$P(C | \text{Blue}) = \frac{P(\text{Blue} | C)P(C)}{P(\text{Blue})}$$

First calculate

$$\begin{aligned}P(\text{Blue}) &= P(\text{Blue} \mid A)P(A) + P(\text{Blue} \mid B)P(B) + P(\text{Blue} \mid C)P(C) \\&= \frac{3}{10} \times \frac{1}{4} + \frac{2}{5} \times \frac{1}{2} + \frac{2}{15} \times \frac{1}{4} \\&= \frac{3}{40} + \frac{1}{5} + \frac{1}{30} = \frac{37}{120}\end{aligned}$$

Now,

$$P(C \mid \text{Blue}) = \frac{\frac{2}{15} \times \frac{1}{4}}{\frac{37}{120}} = \frac{4}{37}$$

Final Answer

$$P(C \mid \text{Blue}) = \frac{4}{37} \approx 0.108$$

Probability = 10.8%

```
# Bayes Theorem: Probability that the blue car came from Section C

# Section selection probabilities
P_A = 1/4
P_B = 1/2
P_C = 1/4

# Blue car probabilities in each section
P_Blue_A = 3/10
P_Blue_B = 6/15
P_Blue_C = 2/15

# Total probability of selecting a blue car
P_Blue = (P_Blue_A * P_A) + (P_Blue_B * P_B) + (P_Blue_C * P_C)

# Bayes Theorem
P_C_given_Blue = (P_Blue_C * P_C) / P_Blue

# Display results
print("P(Blue) =", P_Blue)
print("P(C | Blue) =", P_C_given_Blue)
print("Probability (%) =", P_C_given_Blue * 100)
```

```
... P(Blue) = 0.30833333333333335
     P(C | Blue) = 0.1081081081081081
     Probability (%) = 10.81081081081081
```

QUESTION.2

Examine the performance of a machine-learning model using its training and validation loss values over multiple epochs:

1. (a) Epoch 1: Training Loss = 2.8, Validation Loss = 2.4
2. (b) Epoch 2: Training Loss = 2.5, Validation Loss = 2.2
3. (c) Epoch 3: Training Loss = 2.2, Validation Loss = 2.0
4. (d) Epoch 4: Training Loss = 2.0, Validation Loss = 2.1
5. (e) Epoch 5: Training Loss = 1.8, Validation Loss = 2.3
6. (f) Epoch 6: Training Loss = 1.6, Validation Loss = 2.6
7. (g) Epoch 7: Training Loss = 1.5, Validation Loss = 2.9
8. (h) Epoch 8: Training Loss = 1.3, Validation Loss = 3.2
9. i) Identify the epoch at which the model begins to overfit.
10. ii) Suggest two effective techniques to reduce overfitting in this model.
11. iii) Discuss how overfitting affects the model's generalization ability on unseen data.

ANSWER:

Given:

Epoch	Training Loss	Validation Loss
1	2.8	2.4
2	2.5	2.2
3	2.2	2.0
4	2.0	2.1
5	1.8	2.3
6	1.6	2.6
7	1.5	2.9
8	1.3	3.2

i) Identify the epoch where overfitting starts

Observation:

- Training loss continuously decreases.
- Validation loss decreases until **Epoch 3**.
- At **Epoch 4**, validation loss starts increasing.

Therefore, **Overfitting starts at Epoch 4.**

ii) Two techniques to reduce overfitting

1. Early Stopping

- Stop training once validation loss begins to increase.
- Prevents the model from memorizing the training data.

2. Regularization (L1/L2 or Dropout)

- Adds penalties to large weights.
- Dropout randomly disables neurons during training.
- Helps the model generalize better.

iii) Effect of overfitting on unseen data

When overfitting occurs,

- The model memorizes the training data.
- It performs very well on training data.
- Performance on unseen test data decreases.
- Prediction accuracy becomes poor.
- Generalization ability is reduced.

Conclusion

The model should stop training around Epoch 3 or 4 using Early Stopping.

```

import matplotlib.pyplot as plt
epochs = [1, 2, 3, 4, 5, 6, 7, 8]
training_loss = [2.8, 2.5, 2.2, 2.0, 1.8, 1.6, 1.5, 1.3]
validation_loss = [2.4, 2.2, 2.0, 2.1, 2.3, 2.6, 2.9, 3.2]
overfit_epoch = None
for i in range(1, len(epochs)):
    if validation_loss[i] > validation_loss[i-1] and training_loss[i] < training_loss[i-1]:
        overfit_epoch = epochs[i]
        break
print("Epoch\tTraining Loss\tValidation Loss")
for i in range(len(epochs)):
    print(f"{epochs[i]}\t{training_loss[i]}\t{validation_loss[i]}")
print("\nModel begins to overfit at Epoch:", overfit_epoch)
print("\nTechniques to reduce overfitting:")
print("1. Early Stopping")
print("2. Regularization (Dropout/L2)")
print("\nEffect of Overfitting:")
print("The model performs well on training data but poorly on unseen data.")
plt.figure(figsize=(8,5))
plt.plot(epochs, training_loss, marker='o', linewidth=2, label='Training Loss')
plt.plot(epochs, validation_loss, marker='s', linewidth=2, label='Validation Loss')
plt.axvline(x=overfit_epoch, linestyle='--', label='Overfitting Starts')
plt.scatter(overfit_epoch, validation_loss[overfit_epoch-1], s=100)
plt.title("Training Loss vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.xticks(epochs)
plt.grid(True)
plt.legend()
plt.show()

```

RESULT:

```

... Epoch  Training Loss  Validation Loss
1    2.8      2.4
2    2.5      2.2
3    2.2      2.0
4    2.0      2.1
5    1.8      2.3
6    1.6      2.6
7    1.5      2.9
8    1.3      3.2

```

Model begins to overfit at Epoch: 4

Techniques to reduce overfitting:

1. Early Stopping
2. Regularization (Dropout/L2)

Effect of Overfitting:

The model performs well on training data but poorly on unseen data.



QUESTION.3

Examine the performance of a machine-learning model using training and validation accuracy over multiple epochs:

- (a) Epoch 1: Training Accuracy = 60%, Validation Accuracy = 58%
 - (b) Epoch 2: Training Accuracy = 65%, Validation Accuracy = 63%
 - (c) Epoch 3: Training Accuracy = 70%, Validation Accuracy = 68%
 - (d) Epoch 4: Training Accuracy = 75%, Validation Accuracy = 72%
 - (e) Epoch 5: Training Accuracy = 80%, Validation Accuracy = 73%
 - (f) Epoch 6: Training Accuracy = 85%, Validation Accuracy = 72%
 - (g) Epoch 7: Training Accuracy = 88%, Validation Accuracy = 70%
 - (h) Epoch 8: Training Accuracy = 92%, Validation Accuracy = 68%
- i) Identify the epoch at which the model starts overfitting.
- ii) Explain why validation accuracy decreases despite training accuracy increasing.
- iii) Suggest two methods to improve validation performance.

ANSWER:

Epoch	Training Accuracy	Validation Accuracy
1	60%	58%
2	65%	63%
3	70%	68%
4	75%	72%
5	80%	73%
6	85%	72%
7	88%	70%
8	92%	68%

i) Identify the epoch where overfitting starts

Observation:

- Training accuracy keeps increasing.
- Validation accuracy increases until **Epoch 5**.
- From **Epoch 6**, validation accuracy decreases.

Therefore, **Overfitting starts at Epoch 6.**

ii) Why does validation accuracy decrease while training accuracy increases?

This happens because

- The model begins memorizing the training data instead of learning general patterns.
- It becomes too specific to the training dataset.
- Hence, it cannot predict new data accurately.
- Training accuracy increases, but validation accuracy decreases

iii) Suggest techniques to improve model performance

Two suitable methods are:

1. Early Stopping

Stop training once validation accuracy begins to decrease.

2. Regularization / Dropout

Reduce model complexity and prevent overfitting.

Other acceptable techniques:

- Data Augmentation
- Cross Validation
- Increase Training Data
- Reduce Model Complexity

```

import matplotlib.pyplot as plt
epochs = [1, 2, 3, 4, 5, 6, 7, 8]
training_accuracy = [60, 65, 70, 75, 80, 85, 88, 92]
validation_accuracy = [58, 63, 68, 72, 73, 72, 70, 68]
plt.figure(figsize=(8,5))
plt.plot(epochs, training_accuracy, marker='o', linewidth=2, label='Training Accuracy')
plt.plot(epochs, validation_accuracy, marker='s', linewidth=2, label='Validation Accuracy')
plt.axvline(x=6, color='red', linestyle='--', label='Overfitting Starts')
plt.scatter(6, 72, color='red', s=100)
plt.title("Training Accuracy vs Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.xticks(epochs)
plt.grid(True)
plt.legend()
plt.show()

```

...

